



ELSEVIER

Information Processing Letters 72 (1999) 65–70

Information
Processing
Letters

www.elsevier.com/locate/ipl

Very fast and simple approximate string matching[☆]

Gonzalo Navarro^{*}, Ricardo Baeza-Yates¹

Department of Computer Science, University of Chile, Blanco Encalada 2120, Santiago, Chile

Received 8 April 1998; received in revised form 29 June 1999

Communicated by D. Gries

Abstract

We improve the fastest known algorithm for approximate string matching, which can be used only for low error levels. By using a new method to verify potential matches and a new optimization technique for biased texts (such as English), the algorithm also becomes the fastest one for medium error levels. This includes most of the interesting cases in this area. © 1999 Published by Elsevier Science B.V. All rights reserved.

Keywords: Approximate matching; Text searching; Information retrieval

1. Introduction

Approximate string matching is one of the main problems in classical string algorithms, with applications to text searching, computational biology, pattern recognition, etc. The problem can be formally stated as follows: given a (long) text of length n , a (short) pattern of length m , and a maximal number k of errors allowed, find all text positions that match the pattern with up to k errors. The allowed errors are character insertions, deletions and substitutions. Text and pattern are sequences of characters from an alphabet of size σ . We call $\alpha = k/m$ the *error ratio* or *error level*. In all the applications of the problem the error levels of interest are low, typically 0.1 to 0.3.

We focus on on-line algorithms, where the classical solution, involving dynamic programming, is $O(mn)$

worst-case time [9]. Many algorithms improved later the classical algorithm (see [6] for a recent survey).

For low error levels, the fastest known algorithm is [12,3,2]. It is a filtration algorithm, i.e., it quickly discards large text areas that cannot contain a match and later verifies candidate areas using a classical algorithm. As all filtration algorithms, it ceases to work well for moderate error levels. In this paper, we use a new technique to verify potential matches developed in [7]. We also improve the filter on biased texts such as natural language. The result is that the algorithm keeps its effectiveness for moderate error levels as well, becoming by far the fastest algorithm for all the interesting cases of approximate string matching.

All the experimental results of this paper were obtained on a Sun UltraSparc-1 running Solaris 2.5.1, with 32 Mb of RAM, which is a 32-bit machine. We tested random text with $\sigma = 32$, and lower-case English text. The patterns were randomly selected from the text (at word beginnings in the English case). Except when otherwise stated, each data point

[☆] This work has been supported in part by FONDECYT grant 1990627.

^{*} Corresponding author. Email: gnavarro@dcc.uchile.cl.

¹ Email: rbaeza@dcc.uchile.cl.

was obtained by averaging the Unix's user time over 50 trials on 10 megabytes of text. We present all the times in seconds.

2. The original algorithm

The original idea of this algorithm was first presented in [12], and consists in noticing that any approximate occurrence of a pattern must include some pattern substrings that appear with no errors.

Lemma. *If a pattern is partitioned into $k + 1$ pieces, then at least one of the pieces can be found with no errors in any approximate occurrence of the pattern.*

This property is easily verified by considering that k errors cannot alter all $k + 1$ pieces of the pattern, so at least one of the pieces must appear unaltered. This reduces the problem of approximate string searching to a problem of multipattern exact search plus verification of potential matches. That is, we split the pattern in $k + 1$ pieces and search all of them in parallel with a multipattern exact search algorithm (i.e., allowing no errors). Each time we find a piece in the text, we verify a neighborhood to determine if the complete pattern appears with k errors or less.

In the original proposal [12], a variant of the Shift-Or algorithm [1] was used for multipattern search. To search r patterns of length m' , this algorithm is $O(rm'n/w)$, where w is the length in bits of the computer word. Since in this case $r = k + 1$ and $m' = \lfloor m/(k + 1) \rfloor$, the search cost is $O(mn/w)$. Later, in [3], the use of a multipattern extension of an algorithm of the Boyer-Moore (BM) family was proposed.

In [2] we designed an extension of the BM-Sunday algorithm [10] to handle multipattern search. The BM-Sunday algorithm is very simple, extremely fast and well suited to short patterns. It is based in sliding a window over the text and checking if the pattern matches the window. To shift the window a table d is computed which for each character c tells how much can be the window displaced if the text character following the window is c . Our adaptation of BM-Sunday to multipattern search splits the pattern in pieces of length $\lfloor m/(k + 1) \rfloor$ and $\lceil m/(k + 1) \rceil$ and forms a tree with the pieces. We also build a

pessimistic table d with all the pieces (the longer pieces are pruned to build the table). This table contains, for each character, the smallest shift allowed among all the pieces. Now, at each text position, we enter in the tree using the text characters from that position onward. If we end up in a leaf, we found a piece, otherwise we did not. In any case, we use the table d to shift to the next text position.

Each time a piece is found we check a neighborhood for a complete match. Suppose we find at text position i the end of a match for the subpattern ending at position j in the pattern. Then, the potential match must be searched in the area between positions $i - j + 1 - k$ and $i - j + 1 + m + k$ of the text, an $(m + 2k)$ -wide area. This checking must be done with an algorithm resistant to high error levels, such as dynamic programming.

This algorithm is the fastest in practice when the total number of verifications triggered is low enough, in which case the search cost is close to $O(kn/m) = O(\alpha n)$. We now determine when the total amount of work due to verifications does not dominate over the search time.

An exact pattern of length ℓ appears in random text with probability $1/\sigma^\ell$. In our case, this is $1/\sigma^{\lfloor m/(k+1) \rfloor} \approx 1/\sigma^{1/\alpha}$. Since the cost of verifying a potential match using dynamic programming is $O(m^2)$, and since there are $k + 1 \approx m\alpha$ pieces to search, the total cost for verifications is $m^3\alpha/\sigma^{1/\alpha}$. This cost must be $O(\alpha)$ so that it does not affect the total cost of the algorithm. This happens for $\alpha < 1/(3\log_\sigma m)$. On English text, we found empirically the limit $\alpha < 1/5$ for $m \leq 30$.

Compared to the original proposal of [12], the use of the Sunday algorithm is less flexible because it cannot search for *extended patterns* (i.e., patterns with sets of characters, wild cards, etc.). However, in [8] the Sunday algorithm was replaced with another one based on bit-parallel suffix automata. The resulting algorithm has almost the same performance and is able to search some extended patterns. However, we focus in simple patterns in this work and keep using the Sunday algorithm.

3. A new verification technique

In the original algorithm the whole pattern is checked each time a piece is found. We presented in [7] a dif-

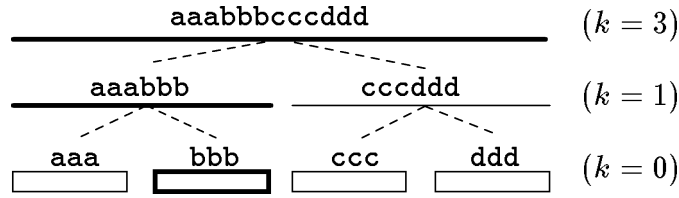


Fig. 1. The hierarchical verification method. The boxes (leaves) are the elements that are really searched, and the root represents the whole pattern. A complete root-to-leaf path must match in any occurrence of the complete pattern. If the bold box is found, all the bold lines may be verified.

ferent verification technique, whose idea is to try to quickly determine that the match of the small piece is not in fact part of a complete match. We now explain the new technique and adapt it to our search algorithm.

To explain the verification technique, a stronger version of the lemma must be used, which was proved in [7].

Stronger Lemma. *If $\text{segm} = \text{Text}[a..b]$ matches pat with k errors, and $\text{pat} = p_1 \dots p_j$ (a concatenation of subpatterns), then segm includes a substring that matches at least one of the p_i 's, with $\lfloor a_i k / A \rfloor$ errors, where the $a_i \geq 0$ are arbitrary and $A = \sum_{i=1}^j a_i$.*

We first explain the case where $k + 1$ is a power of 2, where we use the lemma with $A = 2$, $j = 2$, $a_1 = a_2 = 1$. That is, we split the pattern in two halves (halving also the number of errors). The stronger lemma states that at least one half must match with $\lfloor k/2 \rfloor$ errors. We recursively continue with this splitting, halving the subpatterns and the number of errors, until we reach the pieces that are to be searched directly (with no errors). This defines a tree, where the leaves represent the pattern pieces that are searched directly with zero errors, the internal nodes are larger subpatterns used for intermediate verification, and the root corresponds to the complete pattern.

The search algorithm is the same as before, but the verification is more complex. Imagine that a given leaf of the tree is found. Instead of immediately checking the root of the tree for an occurrence with k errors, the parent of the leaf found is checked (with $k = 1$) in a neighborhood of the area where the leaf was found. That parent is approximately twice the length of the leaf pattern. Only if the parent node finds the longer pattern with 1 error does it report the occurrence to its

parent, who checks a longer neighborhood with $k = 2$ or 3 errors, depending on the case. This continues recursively until the root of the tree. The occurrences reported by the root are the final answers.

This construction is correct because the lemma applies to each level of the tree, i.e., any occurrence reported by the root node *must* include an occurrence reported by one of the two halves, so we search both halves. The argument applies then recursively to each half.

Fig. 1 illustrates this concept. If we search the pattern "aaabbbccdd" with three errors in the text "xxxbbbxxxxx" and split the pattern into four pieces, the piece "bbb" will be found in the text. In the original approach, we would verify the complete pattern in the text area, while with the new approach we verify only if its parent "aaabbb" appears with one error and immediately determine that there cannot be a complete match.

If $k + 1$ is not a power of two, we try to build the tree as balanced as possible (to avoid verifying a very long parent because of a very short child). For instance, if $k = 4$ ($k + 1 = 5$), we partition the tree into, say, a left child with three pieces and a right child with two pieces. We then search the left subtree with $\lfloor 3k/5 \rfloor$ errors and the right one with $\lfloor 2k/5 \rfloor$ errors. Continuing with this policy we arrive at the leaves, which are searched with $\lfloor 4/5 \rfloor = 0$ errors each, as expected.

Although when there are few matches (i.e., low error level) the old and new methods behave similarly, there is an important difference for medium error levels: the new algorithm is more tolerant to errors.

If we use the hierarchical verification algorithm, the analysis of [7] can be easily extended [5] to show that the verification cost per piece is $\ell^2 / \sigma^{\lfloor m/(k+1) \rfloor}$. Since $\ell = m/(k + 1) \approx 1/\alpha$, and since there are $k + 1 \approx$

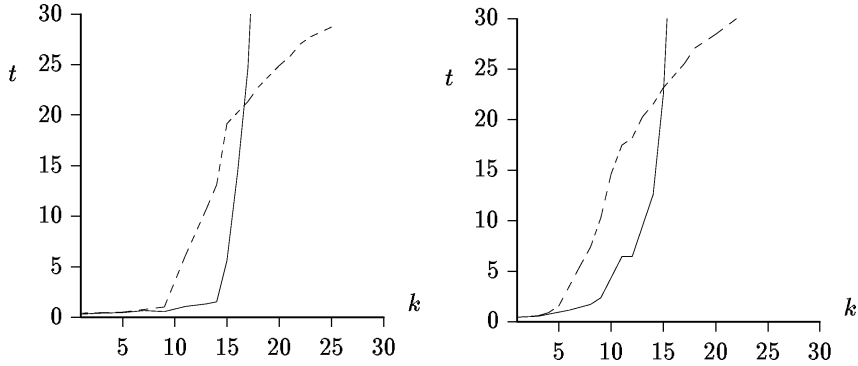


Fig. 2. The old (dashed line) versus the new (solid line) verification techniques. We use $m = 30$ and show random (left) and English text (right).

$m\alpha$ pieces to search, the total cost for verifications is $m/(\alpha\sigma^{1/\alpha})$. This cost must be $O(\alpha)$ so that it does not affect the total cost of the algorithm. This happens for $\alpha < 1/\log_\sigma m$.

Fig. 2 illustrates the improvement obtained. As can be seen, on random text the new method works well up to $\alpha = 1/2$, while the previous works well up to $\alpha = 1/3$. On the other hand, after that point the verifications cost much more than in the original method. This is because of the hierarchy of verifications that is carried out for most text positions when the error level is high. On the other hand, it is hard to improve the barrier of $\alpha < 1/2$ with this method, since at this point we are searching for single characters and performing a verification each time some of the characters are found in the text (which is very probable).

4. Optimizing the partition

When splitting the pattern, we are free to determine the $k + 1$ pieces as we like. This can be used to minimize the expected number of verifications when the letters of the alphabet do not have the same probability of occurrence (e.g., in English text).

For example, imagine that $Pr('e') = 0.3$ and $Pr('z') = 0.003$. If we search for "eeez" it is better to partition it as "eee" and "z" (with probabilities 0.0027 and 0.003, respectively) rather than "ee" and "ez" (with probabilities 0.09 and 0.0009, respectively). More generally, given that the probability of a sequence is the product of the individual letter probabilities, we want a partition that minimizes the sum of

the probabilities of the pieces (which is directly related to the number of verifications to perform).

A dynamic programming algorithm to optimize the partition of $pat[0..m - 1]$ follows. Let

$$R[i, j] = \prod_{r=i}^{j-1} Pr(pat[r])$$

for every $0 \leq i \leq j \leq m$. It is easy to see that R can be computed in $O(m^2)$ time since

$$R[i, j + 1] = R[i, j] \times Pr(pat[j]).$$

Using R we build two matrices, namely

$P[i, k]$ = sum of the probabilities of the pieces in the best partition for $pat[i..m - 1]$ with k errors.

$C[i, k]$ = where the next piece must start in order to obtain $P[i, k]$.

This takes $O(m^2)$ space. The following algorithm computes the optimal partition in $O(m^2k)$ time. The final probability of verification is $P[0, k]$. The pieces start at $\ell_0 = 0$, $\ell_1 = C[\ell_0, k]$, $\ell_2 = C[\ell_1, k - 1]$, ..., $\ell_k = C[\ell_{k-1}, 1]$.

```

for ( $i = 0$ ;  $i < m$ ;  $i++$ )
   $P[i, 0] = R[i, m]$ ;
   $C[i, 0] = m$ ;
for ( $r = 1$ ;  $r \leq k$ ;  $r++$ )
  for ( $i = 0$ ;  $i < m - r$ ;  $i++$ )
     $P[i, r] = \min_{j \in i+1..m-r} (R[i, j] + P[j, r - 1])$ ;
     $C[i, r] = j$  that minimizes
      the expression above;
```

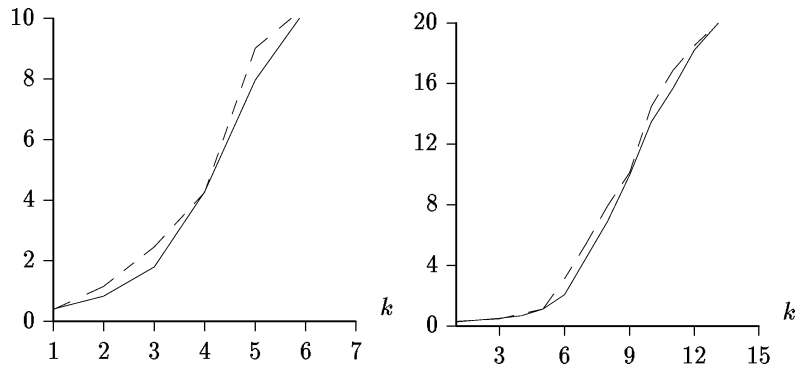


Fig. 3. The optimized (solid line) and the normal splitting (dashed line), for $m = 10$ (left) and 30 (right) on English text.

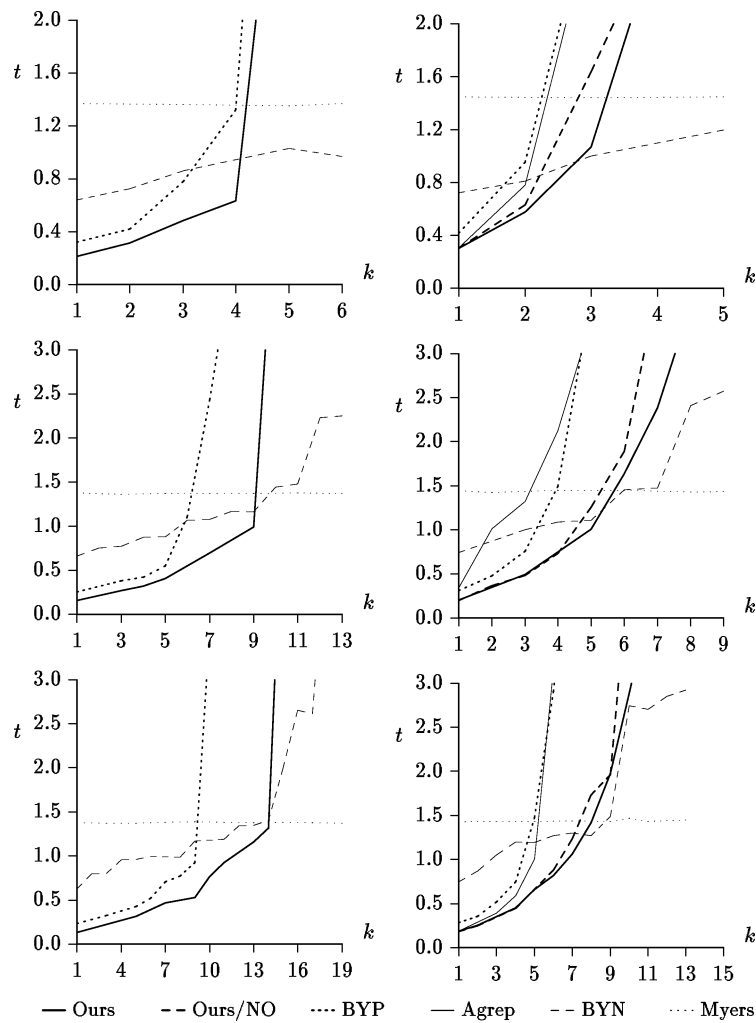


Fig. 4. Experimental results for random text (left) and English text (right). From top to bottom $m = 10, 20$ and 30.

As we presented the optimization, the obtained speedup is very modest and even counterproductive in some cases. This is because we consider only the probability of verifying. The search time of the extended Sunday algorithm degrades as the length of the shortest piece is reduced, as it happens in an uneven partition. We consider in fact a cost model that is closer to the real search cost: we optimize $1/(\text{minimum length}) + Pr(\text{verifying}) \times m^2$. Better estimates are possible and we are working on them.

Fig. 3 shows experimental results comparing the normal versus the optimized partitioning algorithms. We repeated this experiment 100 times instead of 50 because of its very high variance. This experiment is only run on English text since it has no effect on random text. Both cases use the original verification method, not the hierarchical one. As can be seen, the achieved improvements are especially noticeable in the intermediate range of errors.

5. Experimental comparison

In this section we experimentally compare the old and new algorithms against the fastest algorithms we are aware of. We compare only the fastest algorithms, leaving aside many others that are not competitive in the range of parameters we study here. The algorithms included in this comparison are: BYP [3] (i.e., the original version of this algorithm), BYN [2] and Myers [4] (the other fastest algorithms, based on bit-parallelism), and Ours (our modification to BYP with hierarchical verification). The code is from the authors in all cases.

On English text we add two extra algorithms: Agrep [11] (the fastest known approximate search software) and a version of our algorithm that includes the splitting optimization. On English text, the code “Ours” corresponds to our algorithm with hierarchical verification and splitting optimization, while

“Ours/NO” shows hierarchical verification and no splitting optimization.

As seen in Fig. 4, for $\sigma = 32$ our new algorithm is more efficient than any other for $\alpha < 1/2$, while for English text it is the fastest for $\alpha < 1/3$. Notice that although Agrep is normally faster than BYP (i.e., the original version of this technique), we are faster than Agrep with the hierarchical verification, and the splitting optimization improves a little over this.

References

- [1] R. Baeza-Yates, G. Gonnet, A new approach to text searching, *Comm. ACM* 35 (10) (1992) 74–82.
- [2] R. Baeza-Yates, G. Navarro, Faster approximate string matching, *Algorithmica* 23 (2) (1999) 127–158. Preliminary version in: *Proc. CPM'96*.
- [3] R. Baeza-Yates, C. Perleberg, Fast and practical approximate pattern matching, *Inform. Process. Lett.* 59 (1996) 21–27.
- [4] G. Myers, A fast bit-vector algorithm for approximate pattern matching based on dynamic programming, in: *Proc. CPM'98*, *Lecture Notes in Computer Sci.*, Vol. 1448, Springer, Berlin, 1998, pp. 1–13.
- [5] G. Navarro, Approximate text searching, Ph.D. Thesis, Department of Computer Science, University of Chile, December 1998. Tech. Report TR/DCC-98-14.
- [6] G. Navarro, A guided tour to approximate string matching, Technical Report TR/DCC-99-5, Department of Computer Science, University of Chile, 1999. Submitted. <ftp://ftp.dcc.uchile.cl/pub/users/gnavarro/survasm.ps.gz>.
- [7] G. Navarro, R. Baeza-Yates, Improving an algorithm for approximate string matching, 1998, submitted.
- [8] G. Navarro, M. Raffinot, A bit-parallel approach to suffix automata: Fast extended string matching, in: *Proc. CPM'98*, *Lecture Notes in Computer Sci.*, Vol. 1448, Springer, Berlin, 1998, pp. 14–33.
- [9] P. Sellers, The theory and computation of evolutionary distances: pattern recognition, *J. Algorithms* 1 (1980) 359–373.
- [10] D. Sunday, A very fast substring search algorithm, *Comm. ACM* 33 (8) (1990) 132–142.
- [11] S. Wu, U. Manber, Agrep – a fast approximate pattern-matching tool, in: *Proc. of USENIX Technical Conference*, 1992, pp. 153–162.
- [12] S. Wu, U. Manber, Fast text searching allowing errors, *Comm. ACM* 35 (10) (1992) 83–91.